

Hand Bone Image Segmentation

CV-16 SOTA조

김동욱, 김세린, 김재진, 이재건, 박정욱, 황은섭

1. 프로젝트 소개

1.1 개요

프로젝트명: Hand Bone Image Segmentation

프로젝트 기간: 2024.11.13 10:00 ~ 2024.11.28 19:00

실험 환경: V100 Linux 서버 4개

Tools: Visual Studio Code, Github, Slack, smp, torchseg, mmsegmentation

커뮤니케이션 및 협업: Slack, Zoom, Notion, Kakaotalk

1.2 프로젝트 목표

- 사람의 Hand bone X-Ray 이미지 데이터를 받아 각 이미지를 Semantic Segmentation을 통해 각 픽셀이 어떤 뼈의 class인지를 예측하는 인공지능 만들기입니다.

1.3 Dataset

- Train 데이터 800장, Test 데이터 288장으로 구성된 Hand bone X-Ray 이미지 데이터입니다.
- 각 이미지는 2048 x 2048 사이즈를 가지고 한 사람 당 왼손, 오른손 짝이 지어져 있습니다.

- 이미지마다 총 29개의 Class가 있습니다.

2. 프로젝트 역할 분담

이름	공통	
김동욱	모델 실험, 레포트 작성	Streamlit 구현, copy_paste 증강 구현, 앙상블 실험
김세린		
김재진		
이재건		코드환경 및 streamlit 초기 설정, Notion 및 Github 협업 지도, wandb설정.
박정욱		
황은섭		EDA, augmenataiton 실험, loss 실험, LoRA 실험

3. 실험 내용

3.1 EDA

이번 프로젝트의 경우 이미지가 외부에 노출되면 안되기에 글로 설명을 하겠습니다.

3.1.1 각 Class의 비율

손가락 뼈는 손가락의 끝 쪽으로 갈 수록 뼈의 크기가 작아집니다.

손등 부분은 겹치는 구간이 많고 크기도 작은 편입니다. 눈으로 확인해도 구별이 어렵습니다. 이번 프로젝트에서 모델들이 제일 학습하기 어려워하는 class 입니다.

팔 뼈는 크기가 제일 크고, 큰 분산을 가집니다. X-ray를 찍을 때 사람마다 손의 위치가 다르기 때문에

생각됩니다.

3.1.2 Multi Label

다른 뼈들도 겹치는 부분이 있지만 상대적으로 Pisiform과 Triquetrum, Trapezoid와 Trapezium가 겹치는 부분이 크고 많았습니다.

3.1.3 Meta data

dataset에 있는 meta_data 파일을 통해 분석했습니다. train dataset과 test dataset이 구별되지 않은 채로 544명의 키, 몸무게, 성별, 나이에 대한 정보가 들어있습니다.

강의에서 제공한 그래프로 나이에 따른 성별, 몸무게, 신장의 분포를 얻을 수 있었습니다.

추가로 진행한 Meta data로는 성별에 따른 손 크기 분석, 나이 분포 분석이 있습니다. 성별에 따른 손 크기 분석은 성별에 따라 손의 크기가 명확히 구분할 수 있다면 성별을 prompt로 사용할 수 있을 것 같아 진행했습니다. 남자의 손 크기가 여자보다는 컸지만, 그 차이가 크지 않아 prompt로 사용하는 것은 효과가 크지 않을 것이라 예상되었습니다. 나이는 19세부터 60대 후반까지 있었으며, 20~30대가 제일 많았습니다. 카메라와 손의 거리가 이미지마다 달라 몸무게와 신장, 성별을 활용하는 것은 어렵다고 판단했습니다.

3.1.4 데이터셋 확인

데이터를 확인하면서 발견한 특징입니다.

- X-ray를 찍으면서 위쪽 부분이나 좌측에 흰 부분이 보이는 이미지
- 손톱에 무언가가 붙어있는 손 이미지
- 손의 방향이 다른 이미지
- 반지로 보이는 무언가가 손가락에 끼어있는 이미지

- 손가락 뼈가 특이한 이미지
- 손등 부분에 흰머리 같은 자국이 보이는 이미지
- 손목 부근 뼈를 고정하기 위한 의료 도구가 있는 이미지

반지가 있거나, 의료 도구가 있는 이미지는 이상 데이터로도 분류할 수 있을 정도였습니다. 손의 방향이 다른 이미지는 수가 학습하기에 충분할 정도로 많았습니다.

3.1.5 손목, 손등, 손가락 뼈 위치 분포 시각화

손가락 뼈의 위치는 손의 방향에 따라 2가지로 나뉘었습니다. 손등과 손목은 모두 특정 부분에 위치했습니다. 여기서 얻은 정보는 Crop 실험을 진행할 때 위치를 알아내기 위해 사용했습니다.

3.2 augmentation 실험

Transformation

	Dice Score
Default	0.9385
Horizontal Flip	0.9382
Rotate	0.9382
RandomBrightnessContrast	0.9373
Gaussian Noise	0.9384
HFlip + Rotate + RandomBrightnessContrast	0.9371
Grid Distort	0.5548

- Flip, Rotate 등의 증강을 추가해서 실험을 해본 결과, 큰 성능 향상을 보지는 못했습니다. 대부분의 X-Ray 이미지는 촬영 시, 유사한 형태와 위치에서 촬영을 하기에 위의 증강들이 큰 효과를 보지 못한 것으로 생각합니다.

Resize 실험

	Dice Score
Resize: 512	0.9434
Resize: 1024	0.9613
Resize: 1536	0.9670
Resize: 2048	0.9687

- 이미지 사이즈 실험 결과, 이미지 사이즈가 클 수록 성능이 좋게 나오는 것을 확인 할 수 있습니다. 해상도가 높을 수록 객체의 디테일한 정보와 경계를 더 잘 보존하고, 원본 이미지의 해상도의 변화가 없기에 Resize를 통한 정보의 손실도 없기에 2048로 Resize한 것이 가장 좋은 성능을 보인 것 같습니다.

Preprocessing

- Hand bone 부분과 각 class의 경계선 부분을 강조하기 위해 Clahe와 Sharpen 증강을 Preprocessing으로 각각 사용해 보았습니다.

	Dice Score
Default	0.9670
Sharpen	0.9661
Clahe	0.9640

- 실험 결과로 Sharpen과 Clahe 증강을 사용하는 것보다는 원본 이미지 자체를 사용한 것이 가장 성능이 좋았습니다.

손등 뼈를 위한 처리

	Dice Score
손등 뼈 부분만 Crop	0.9718

이미지의 전체 학습 후 손등 부분만 추가학습	0.7880
Copy and Paste	0.9730

20%의 확률로 손등 뼈의 GT부분만 학습하는 증강을 추가하여 학습해보기도 했습니다. valid: 0.9738 -> 0.9778, test: 0.9718 -> 0.9682로 valid에서는 성능이 증가했지만 test에서는 많은 성능 하락이 있었습니다. valid과 test의 성능 차이가 큰 이유는 valid dataset 학습에 적용했던 증강이 그대로 적용되고 있기 때문이었습니다. 여러 augmentation도 추가해보았지만, 기존의 성능보다 뛰어나지지는 않았습니다.

channel 1

이미지는 RGB가 아니고 채널이 1 개인 2048x2048의 크기를 가지고 있습니다. 베이스라인 코드는 이미지를 3개의 채널로 학습을 합니다. 채널을 1개로 유지하며 학습을 하면 메모리 사용량의 감소, 학습 속도 단축, 성능 변화가 예상되어 실험을 진행해보았습니다.

3.n Loss 실험

아래 표에서 default 모델은 아래와 같이 설정했습니다.

- 모델: DeepLabV3+
- 인코더: efficient-b0
- Augmentation: resize 512, ch 1
- scheduler: CosineAnnealingLR
- epoch: 40

	BCE Loss	Focal + Dice	Focal0.7 + Dice0.3	BCE + Dice
valid	0.9423	0.9489	0.9498	0.9442
test	0.9384	0.9449	0.9456	0.9379

가장 성능이 많이 오른 Combine Loss는 Focal Loss에 weight를 0.7, Dice에 0.3을 주었을 때 였습니다.

이때 학습에서 성능이 오르는 속도도 확연하게 빨랐습니다.

IoU Loss도 있었습니다. DICE와 비교해서 class가 위치하는 것보단 위치하지 않는 것에 좀 더 초점을 맞추는 것으로 알고 있어서 DICE보다 성능이 떨어질 것으로 예상하였고, 시간 관계상 실험은 하지 않았습니다.

3.n threshold 분석

test 과정에서 threshold를 다르게 하면 성능이 얼마나 달라지는지 확인해보았습니다.

default threshold는 0.5입니다.

threshold	0.01	0.4	0.5	0.6	0.9	0.9999
test	0.9678	0.9724	0.9725	0.9725	0.9719	0.9587

threshold를 0.4와 0.6으로 test 결과를 보았을 때, 성능의 차이가 거의 없었습니다. 이를 이상하게 여겨 0.9의 threshold로 test를 해보았는데, 약간의 성능이 하락만 모습이었습니다. threshold가 0.01, 0.9999일 때도 확인해보면 threshold가 극단적임에도 성능이 상당히 높은 모습입니다.

threshold를 조정해도 성능의 차이가 미미한 원인을 알기 위해서 모델이 1개의 class에 대해 예측한 결과를 sigmoid 함수를 사용하기 전의 상태로 txt 파일을 얻어 살펴보았습니다. 약 4.5 값일 때 sigmoid의

값이 0.99가 되는데, class의 경계 부분은 대부분 -5 ~ +5의 값을 가지고 있었고 그 외의 부분은 -5보다 더 작거나 +5 보다 더 큰 값을 가지고 있었습니다. 생각보다 sigmoid 이후 0.1 ~ 0.9의 값을 가지는 픽셀들이 매우 적다는 것과, 이 픽셀들이 경계선 부근에만 존재한다는 것을 알 수 있었습니다.

txt 파일을 한 눈에 볼 수 있도록 시각화를 했더니 해당 class 부분만 나타나는 것이 아닌 경계선 부분이 두드러진 모든 손 뼈가 나타났습니다. 한 class에 대해서 예측한 경계선 부근은 다른 class 결과과에서는 매우 낮은 값이 나오는 것을 알았습니다. 이를 통해 모델이 class를 예측하는 과정에서 class마다 독립적으로 예측하는 것이 아니고 상호관계가 있다는 것을 경험할 수 있었습니다.

3.n 앙상블 실험

4. 자체 평가

4.1 잘한 점들

- Github를 통해 코드 공유가 원활히 되었습니다.
- 또한 Notion을 통해 각자의 의견들을 제시 및 공유하고, 실험 결과에 대한 내용도 공유하여 같은 실험을 반복하는 일이 없었습니다.
- Semantic Segmentation 관련 논문을 서칭 및 리뷰하며, 아이디어를 얻으려고 시도하였습니다.

4.2 시도했으나 잘 되지 않았던 것들

- 손등 부분의 성능을 올리기 위해 여러 시도를 해보았지만, 만족할 만한 성능 향상을 이뤄내지 못했습니다.
- 논문 리뷰를 통해 알게 된 PerPixel Contrastive Loss를 시간 관계 상 시도해 보지 못했습니다.
- 프로젝트 막바지에 CRF를 진행하려 했지만, 제대로 구현하지 못하였습니다.

4.3 아쉬웠던 점들

- PerPixel Contrastive Loss, CRF 등 여러 아이디어가 있었지만, 모두 수행해보지 못한 점이 아쉬운 것 같습니다.

- 여러 증강을 시도했지만, 만족할 만한 성능 향상을 이뤄낸 증강을 찾지 못한 점이 아쉽습니다.
- Streamlit을 통해 Ground Truth와 예측 결과를 비교하려 하였지만, 픽셀 별로 구별되기에 눈으로 확인하기 어려워 제대로 사용하지 못한 점이 아쉬운 것 같습니다.
- 메모리가 부족한 부분에 대해서 auto mixed precision 말고 더 알아볼 생각을 못한 것이 아쉽습니다.

4.4 프로젝트를 통해 배운 점 또는 시사점

- 협업 Tool을 좀 더 잘 쓰게 되었음.
- Segmentation 을 통해 좀더 다양한 모델을 사용할 수 있게 되었음.
-

개인 회고록

이재건

초기 세팅에 많은 노력을 기울였습니다. 처음 세팅이 잘 이루어져야 이후 빌드 과정이 수월하고 협업도 원활하다고 생각했기 때문입니다. 하지만 여전히 부족한 점이 있었습니다. 예를 들어, 설정(config)을 하나의 파일에 모두 담다 보니 혼란이 생기고 오류가 자주 발생했습니다. 또한, 코드를 모듈화하지 않아 다른 코드와의 충돌을 방지하려는 과정에서 많은 문제가 있었습니다. 처음부터 코드를 모듈화하고 체계적으로 구성했다면 더 효율적이지 않았을까 하는 아쉬움이 남습니다. 그럼에도 불구하고, 세그멘테이션 관련 코딩, GitHub와 Notion을 활용한 협업, 그리고 오류를 통해 배운 점들을 통해 한 단계 더 성장할 수 있었다고 생각합니다.